NAME : KISHORE KUMAR V

REG NO : 192521371

COURSE CODE : ITA 0613

EXP 3: Decision Tree based ID3 Algorithm

Session Topic: Demonstrate the working of the decision tree based ID3 algorithm. Use an appropriate data set for building the decision tree and apply this knowledge to classify a new sample.

Question 1: ID3 Algorithm on the PlayTennis Data Set

Aim:

To implement the ID3 decision tree algorithm from scratch in Python using the PlayTennis training data set, build the decision tree, and use it to classify a new sample.

Algorithm:

1. Start. Load the PlayTennis training data set (14 examples, 4 attributes, 1 class label).
2. Calculate the entropy of the whole data set: Entropy(S) = - Σ p(i) log2 p(i).
3. For every attribute A, calculate Information Gain(S, A) = Entropy(S) - Σ (|Sv| / |S|) × Entropy(Sv).
4. Select the attribute with the highest information gain as the root (decision) node.
5. Create one branch for every value of the selected attribute and split the data accordingly.
6. Repeat steps 2 to 5 recursively for every branch using the remaining attributes, until all examples in a branch belong to one class or no attributes are left.

7. Display the tree and classify the new sample by following the branches from the root to a leaf.
8. Stop.

Python Code:

```
import math
import pandas as pd
from collections import Counter


# PlayTennis training data
rows = [
 ("Sunny","Hot","High","Weak","No"),
 ("Sunny","Hot","High","Strong","No"),
 ("Overcast","Hot","High","Weak","Yes"),
 ("Rain","Mild","High","Weak","Yes"),
 ("Rain","Cool","Normal","Weak","Yes"),
 ("Rain","Cool","Normal","Strong","No"),
 ("Overcast","Cool","Normal","Strong","Yes"),
 ("Sunny","Mild","High","Weak","No"),
 ("Sunny","Cool","Normal","Weak","Yes"),
 ("Rain","Mild","Normal","Weak","Yes"),
```

```
 ("Sunny","Mild","Normal","Strong","Yes"),
 ("Overcast","Mild","High","Strong","Yes"),
 ("Overcast","Hot","Normal","Weak","Yes"),
 ("Rain","Mild","High","Strong","No")]
data = pd.DataFrame(rows,
columns=["Outlook","Temperature",
                              "Humidity","Wind","PlayTennis"])

def entropy(col):
    n = len(col)
    return -sum((c/n) * math.log2(c/n) for c in
Counter(col).values())

def info_gain(df, attr, target):
    total = entropy(df[target])
    weighted = sum((len(s)/len(df)) * entropy(s[target])
                   for _, s in df.groupby(attr))
    return total - weighted

def id3(df, target, attrs):
    if df[target].nunique() == 1:
```

```
        return df[target].iloc[0]
    if not attrs:
        return df[target].mode()[0]
    best = max(attrs, key=lambda a: info_gain(df, a, target))
    tree = {best: {}}
    for value, subset in df.groupby(best):
        rest = [a for a in attrs if a != best]
        tree[best][value] = id3(subset, target, rest)
    return tree

def classify(tree, sample):
    if not isinstance(tree, dict):
        return tree
    attr = next(iter(tree))
    return classify(tree[attr][sample[attr]], sample)

def show(tree, indent=0):
    for attr, branches in tree.items():
        for value, sub in branches.items():
            print("  " * indent + f"{attr} = {value}", end="")
            if isinstance(sub, dict):
```

```
            print()
            show(sub, indent + 1)
        else:
            print(f"  -->  {sub}")

target = "PlayTennis"
attrs = [c for c in data.columns if c != target]

print("Entropy of the data set = %.4f" %
entropy(data[target]))
for a in attrs:
    print(f"Information Gain({a}) = {info_gain(data, a,
target):.4f}")

tree = id3(data, target, attrs)
print("\nDecision Tree:")
show(tree)

new = {"Outlook": "Sunny", "Temperature": "Cool",
"Humidity": "High", "Wind": "Strong"}
print("\nNew sample:", new)
```

```
print("Predicted class:", classify(tree, new))
```

Output:

```
Entropy of the data set = 0.9403
Information Gain(Outlook) = 0.2467
Information Gain(Temperature) = 0.0292
Information Gain(Humidity) = 0.1518
Information Gain(Wind) = 0.0481

Decision Tree:
Outlook = Overcast  -->  Yes
Outlook = Rain
  Wind = Strong  -->  No
  Wind = Weak  -->  Yes
Outlook = Sunny
  Humidity = High  -->  No
  Humidity = Normal  -->  Yes

New sample: {'Outlook': 'Sunny', 'Temperature': 'Cool',
'Humidity': 'High', 'Wind': 'Strong'}
Predicted class: No
```

Result:

The ID3 algorithm was implemented successfully. Outlook has the highest information gain (0.2467), so it became the root node. For the new sample (Sunny, Cool, High, Strong) the tree predicted PlayTennis = No.

Question 2: ID3 Algorithm for Student Result Prediction

Aim:

To use the ID3 decision tree algorithm to build a model that predicts whether a student will pass, based on study hours, attendance and assignment submission.

Algorithm:

1. Start. Create the student data set with attributes StudyHours, Attendance, Assignments and the class label Pass.
2. Compute the entropy of the class label Pass.
3. Compute the information gain of every attribute.
4. Choose the attribute with maximum gain (StudyHours) as the root node.
5. Split the data set on each value of the chosen attribute.
6. Recursively build sub-trees for the impure subsets with the remaining attributes; make a leaf node when a subset is pure.

7. Print the final decision tree and pass a new student record through it to predict the result.
8. Stop.

Python Code:

```
import math
import pandas as pd
from collections import Counter

# Student result data set
rows = [
 ("High","Good","Yes","Yes"),
("High","Good","No","Yes"),
 ("High","Poor","Yes","Yes"),  ("High","Poor","No","No"),

("Medium","Good","Yes","Yes"),("Medium","Good","No",
"No"),
 ("Medium","Poor","Yes","No"),
("Medium","Poor","No","No"),
 ("Low","Good","Yes","No"),
("Low","Good","No","No"),
```

```
 ("Low","Poor","Yes","No"),    ("Low","Poor","No","No"),
 ("High","Good","Yes","Yes"),
("Medium","Good","Yes","Yes")]
df = pd.DataFrame(rows,
columns=["StudyHours","Attendance","Assignments","Pas
s"])

def entropy(s):
    n = len(s)
    return -sum((c/n)*math.log2(c/n) for c in
Counter(s).values())

def gain(d, a, t="Pass"):
    return entropy(d[t]) - sum(len(g)/len(d)*entropy(g[t]) for
_, g in d.groupby(a))

def build(d, attrs, t="Pass"):
    if d[t].nunique() == 1:
        return d[t].iloc[0]
    if not attrs:
        return d[t].mode()[0]
```

```
    best = max(attrs, key=lambda a: gain(d, a))
    return {best: {v: build(g, [a for a in attrs if a != best])
                   for v, g in d.groupby(best)}}

def predict(tree, x):
    while isinstance(tree, dict):
        attr = next(iter(tree))
        tree = tree[attr][x[attr]]
    return tree

def show(tree, ind=0):
    for a, br in tree.items():
        for v, sub in br.items():
            if isinstance(sub, dict):
                print("   " * ind + f"[{a} = {v}]")
                show(sub, ind + 1)
            else:
                print("   " * ind + f"[{a} = {v}] -> Pass = {sub}")

attrs = ["StudyHours", "Attendance", "Assignments"]
print("Entropy(Pass) = %.4f" % entropy(df["Pass"]))
```

```
for a in attrs:
    print("Gain(%s) = %.4f" % (a, gain(df, a)))

tree = build(df, attrs)
print("\nDecision Tree:")
show(tree)

sample = {"StudyHours": "Medium", "Attendance":
"Good", "Assignments": "Yes"}
print("\nNew sample:", sample)
print("Will the student pass?", predict(tree, sample))
```

Output:

```
Entropy(Pass) = 0.9852
Gain(StudyHours) = 0.3806
Gain(Attendance) = 0.1613
Gain(Assignments) = 0.1613

Decision Tree:
[StudyHours = High]
   [Attendance = Good] -> Pass = Yes
```

[Attendance = Poor]

[Assignments = No] -> Pass = No

[Assignments = Yes] -> Pass = Yes

[StudyHours = Low] -> Pass = No

[StudyHours = Medium]

[Attendance = Good]

[Assignments = No] -> Pass = No

[Assignments = Yes] -> Pass = Yes

[Attendance = Poor] -> Pass = No

New sample: {'StudyHours': 'Medium', 'Attendance': 'Good', 'Assignments': 'Yes'}

Will the student pass? Yes

Result:

The decision tree was built using information gain. StudyHours (gain 0.3806) was chosen as the root. The new student (Medium study hours, Good attendance, assignments submitted) was predicted to Pass.

Question 3: ID3 Algorithm for Loan Approval Using a CSV File

Aim:

To read a training data set from a .CSV file, build a decision tree using the ID3 algorithm, find its training accuracy, and classify a new loan applicant.

Algorithm:

1. 1.Start. Write the loan data to a CSV file and read it into a pandas DataFrame.

a. 2.Calculate the entropy of the target attribute Approved.

2. Calculate the information gain of Income, Employment, Credit_History and Property.
3. Select the attribute with the highest gain (Credit_History) as the root node.
4. Partition the data on each value of that attribute and repeat recursively; stop when a partition is pure or use the majority class if no attributes remain.
5. Display the tree and compute the training accuracy by predicting every record.

6. Classify the new applicant using the tree.
7. Stop.

Python Code:

```
import math
import pandas as pd
from collections import Counter

# Loan approval data set (stored as a CSV file, then read back)
csv_text =
"""Income,Employment,Credit_History,Property,Approved
High,Salaried,Good,Urban,Yes
High,Salaried,Bad,Urban,No
Medium,Salaried,Good,Urban,Yes
Low,Self-Employed,Good,Rural,No
Low,Salaried,Good,Rural,Yes
Medium,Self-Employed,Bad,Urban,No
High,Self-Employed,Good,Urban,Yes
Low,Self-Employed,Bad,Rural,No
Medium,Salaried,Good,Rural,Yes
```

```
High,Salaried,Good,Rural,Yes
Medium,Self-Employed,Good,Urban,Yes
Low,Salaried,Bad,Urban,No
High,Self-Employed,Bad,Rural,No
Medium,Salaried,Bad,Rural,No
Low,Self-Employed,Good,Urban,No
High,Salaried,Good,Urban,Yes
"""
open("loan.csv", "w").write(csv_text)
df = pd.read_csv("loan.csv")
target = "Approved"

def entropy(s):
    n = len(s)
    return -sum((c/n)*math.log2(c/n) for c in
Counter(s).values())

def gain(d, a):
    return entropy(d[target]) -
sum(len(g)/len(d)*entropy(g[target]) for _, g in
d.groupby(a))
```

```
def id3(d, attrs):
    if d[target].nunique() == 1:
        return d[target].iloc[0]
    if not attrs:
        return d[target].mode()[0]
    best = max(attrs, key=lambda a: gain(d, a))
    node = {best: {}, "_default": d[target].mode()[0]}
    for v, g in d.groupby(best):
        node[best][v] = id3(g, [a for a in attrs if a != best])
    return node

def predict(tree, x):
    while isinstance(tree, dict):
        attr = [k for k in tree if k != "_default"][0]
        tree = tree[attr].get(x[attr], tree["_default"])
    return tree

def show(tree, ind=0):
    attr = [k for k in tree if k != "_default"][0]
    for v, sub in tree[attr].items():
```

```
        if isinstance(sub, dict):
            print("    " * ind + f"{attr} = {v}:")
            show(sub, ind + 1)
        else:
            print("    " * ind + f"{attr} = {v}: {sub}")

attrs = [c for c in df.columns if c != target]
print("Total records:", len(df))
print("Entropy(Approved) = %.4f" % entropy(df[target]))
for a in attrs:
    print("Gain(%s) = %.4f" % (a, gain(df, a)))

tree = id3(df, attrs)
print("\nDecision Tree:")
show(tree)

correct = sum(predict(tree, r) == r[target] for _, r in
df.iterrows())
print("\nTraining accuracy = %.2f%%" % (100 * correct /
len(df)))
```

```
new = {"Income": "Medium", "Employment": "Self-
Employed",
      "Credit_History": "Good", "Property": "Rural"}
print("New applicant:", new)
print("Loan approved?", predict(tree, new))
```

Output:

```
Total records: 16
Entropy(Approved) = 1.0000
Gain(Income) = 0.1266
Gain(Employment) = 0.1058
Gain(Credit_History) = 0.5488
Gain(Property) = 0.0115

Decision Tree:
Credit_History = Bad: No
Credit_History = Good:
   Income = High: Yes
   Income = Low:
      Employment = Salaried: Yes
      Employment = Self-Employed: No
```

Income = Medium: Yes

Training accuracy = 100.00%

New applicant: {'Income': 'Medium', 'Employment': 'Self-Employed', 'Credit_History': 'Good', 'Property': 'Rural'}

Loan approved? Yes

Result:

The ID3 tree was built from the CSV data with Credit_History as the root node (gain 0.5488) and achieved 100% training accuracy. The new applicant was predicted as Loan Approved = Yes.

Question 4: ID3 (Entropy Based) Decision Tree on the Iris Data Set

Aim:

To demonstrate the working of the entropy / information gain based decision tree (ID3 principle) on the Iris data set using Python, and to classify a new flower sample.

Algorithm:

1. Start. Import the required libraries and load the Iris data set.
2. Split the data into training (70%) and testing (30%) sets.
3. Create a DecisionTreeClassifier with criterion = "entropy" so that nodes are split using information gain.
4. Train the model using the training data.
5. Print the learned decision tree rules.
6. Predict the classes of the test data and calculate the accuracy.
7. Classify a new sample flower using the trained model.

8. Stop.

Python Code:

```
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier,
export_text
from sklearn.metrics import accuracy_score

iris = load_iris()
X = pd.DataFrame(iris.data, columns=iris.feature_names)
y = iris.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y)

# criterion="entropy" makes the tree use information gain
(the ID3 measure)
model = DecisionTreeClassifier(criterion="entropy",
max_depth=3, random_state=42)
```

```
model.fit(X_train, y_train)

print("Decision Tree (entropy based):")
print(export_text(model, feature_names=list(X.columns)))

pred = model.predict(X_test)
print("Test accuracy = %.2f%%" % (100 *
accuracy_score(y_test, pred)))

new = pd.DataFrame([[5.1, 3.5, 1.4, 0.2]],
columns=X.columns)
print("New sample:", new.values.tolist()[0])
print("Predicted class:",
iris.target_names[model.predict(new)[0]])
```

Output:

```
Decision Tree (entropy based):
|--- petal length (cm) <= 2.45
|   |--- class: 0
|--- petal length (cm) >  2.45
|   |--- petal width (cm) <= 1.55
```

```
|   |   |--- petal length (cm) <= 4.95
|   |   |   |--- class: 1
|   |   |--- petal length (cm) >  4.95
|   |   |   |--- class: 2
|   |--- petal width (cm) >  1.55
|   |   |--- sepal length (cm) <= 6.10
|   |   |   |--- class: 2
|   |   |--- sepal length (cm) >  6.10
|   |   |   |--- class: 2
```

Test accuracy = 93.33%

New sample: [5.1, 3.5, 1.4, 0.2]

Predicted class: setosa

Result:

The entropy based decision tree was trained on the Iris data set and obtained a test accuracy of 93.33%. The new sample [5.1, 3.5, 1.4, 0.2] was classified as setosa.

Question 5: ID3 (Entropy Based) Decision Tree for Breast Cancer Classification

Aim:

To build a decision tree using the information gain (entropy) measure on the Breast Cancer data set, evaluate it using a confusion matrix and classification report, and classify a new sample.

Algorithm:

1. Start. Import the libraries and load the Breast Cancer data set.
2. Split the data into training (75%) and testing (25%) sets.
3. Create a DecisionTreeClassifier with criterion = "entropy" and a maximum depth of 3.
4. Fit the model on the training data and print the tree rules.
5. Predict the labels of the test data.
6. Calculate the accuracy, the confusion matrix and the classification report.

7. Classify a new tumour sample and compare it with the actual class.
8. Stop.

Python Code:

```
import pandas as pd
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier,
export_text
from sklearn.metrics import accuracy_score,
confusion_matrix, classification_report

data = load_breast_cancer()
X = pd.DataFrame(data.data, columns=data.feature_names)
y = data.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=1, stratify=y)
```

```
model = DecisionTreeClassifier(criterion="entropy",
max_depth=3, random_state=1)
model.fit(X_train, y_train)

print("Decision Tree (entropy based):")
print(export_text(model, feature_names=list(X.columns)))

pred = model.predict(X_test)
print("Accuracy = %.2f%%" % (100 *
accuracy_score(y_test, pred)))
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, pred))
print("\nClassification Report:")
print(classification_report(y_test, pred,
target_names=data.target_names))

new = X_test.iloc[[0]]
print("Predicted class for a new tumour sample:",
data.target_names[model.predict(new)[0]])
print("Actual class:", data.target_names[y_test[0]])
```

Output:

Decision Tree (entropy based):

```
|--- worst perimeter <= 116.05
|   |--- worst concave points <= 0.15
|   |   |--- worst perimeter <= 102.05
|   |   |   |--- class: 1
|   |   |--- worst perimeter >  102.05
|   |   |   |--- class: 1
|   |--- worst concave points >  0.15
|   |   |--- worst texture <= 26.90
|   |   |   |--- class: 1
|   |   |--- worst texture >  26.90
|   |   |   |--- class: 0
|--- worst perimeter >  116.05
|   |--- worst compactness <= 0.11
|   |   |--- class: 1
|   |--- worst compactness >  0.11
|   |   |--- class: 0
```

Accuracy = 94.41%

Confusion Matrix:

[[47 6]

 [2 88]]

Classification Report:

	precision	recall	f1-score	support
malignant	0.96	0.89	0.92	53
benign	0.94	0.98	0.96	90
accuracy			0.94	143
macro avg	0.95	0.93	0.94	143
weighted avg	0.94	0.94	0.94	143

Predicted class for a new tumour sample: malignant

Actual class: malignant

Result:

The entropy based decision tree classified the Breast Cancer test data with an accuracy of 94.41%. The new tumour sample was correctly predicted as malignant.